Step 1: Query Offline Feature Store via Athena

Data is retrieved from the Athena table associated with `customer-churn-feature-group` to form the training and evaluation matrices.

Create a script named `src/train_xgboost.py` .

Add the initial imports and Athena query logic:

Python

```python
import os
import time
import tarfile
import boto3
import pandas as pd
import numpy as np
import xgboost as xgb
import sagemaker
from sagemaker.feature_store.feature_group import FeatureGroup

# Initialize AWS & SageMaker Sessions
session = sagemaker.Session()
boto_session = boto3.Session(region_name="us-east-1")
sm_client = boto_session.client("sagemaker")
s3_client = boto_session.client("s3")

REGION = "us-east-1"
BUCKET = "company-mlops-churn-production-1066795149900"
FEATURE_GROUP_NAME = "customer-churn-feature-group"
MODEL_PACKAGE_GROUP_NAME = "customer-churn-model-group"

def extract_features_from_offline_store():
    print(f"Querying Offline Store from Feature Group: '{FEATURE_GROUP_NAME}'...")
    fg = FeatureGroup(name=FEATURE_GROUP_NAME, sagemaker_session=session)

    query = fg.athena_query()
    table_name = query.table_name

    query_string = f'SELECT * FROM "{table_name}"'
    output_location = f"s3://{BUCKET}/athena-results/"

    query.run(query_string=query_string, output_location=output_location)
    query.wait()

    df = query.as_dataframe()
```

```
    print(f"Data retrieved successfully. Total rows/cols: {df.shape}")
    return df
```

Step 2: Strip Schema Dependencies & Train XGBoost

To prevent `FailedPrecondition: 400` schema mismatches when invoking SageMaker pre-built containers (which occur when string column headers contain spaces or special characters), we strip metadata columns and pass pure **NumPy matrices (`X.values`)** into `xgb.XGBClassifier` .

Append the data cleaning and training functions to `src/train_xgboost.py` :

Python

```
def train_xgboost_model(df):
    print("--- Cleaning Schema and Engineering Training Sets ---")
    # Remove SageMaker Feature Store audit/metadata columns and primary keys
    metadata_cols = ["EventTime", "write_time", "api_invocation_time", "is_deleted", "custom
    df_clean = df.drop(columns=[col for col in metadata_cols if col in df.columns])

    # Separate target variable (Churn) from features
    X = df_clean.drop(columns=["Churn"])
    y = df_clean["Churn"].astype(int)

    print(f"Training feature dimension count: {X.shape[1]}")

    # CRITICAL: Strip feature column string names by fitting on raw NumPy arrays.
    # This ensures the binary output container artifact expects standard 2D arrays
    # without schema header validation errors on real-time endpoint payloads.
    X_matrix = X.values
    y_vector = y.values

    print("--- Training XGBoost Candidate Model ---")
    model = xgb.XGBClassifier(
        n_estimators=100,
        max_depth=5,
        learning_rate=0.1,
        eval_metric="logloss",
        use_label_encoder=False
    )
    model.fit(X_matrix, y_vector)
    print("XGBoost Training Complete!")
    return model
```

Step 3: Package Artifacts into AWS Serving Format

SageMaker pre-built XGBoost containers require the booster artifact to be saved as `xgboost-model` (or `model.bst`), placed in a directory, and compressed into a `.tar.gz` archive before uploading to Amazon S3.

Append the packaging routine to `src/train_xgboost.py`:

Python

```python
def package_and_upload_model(model):
    print("--- Packaging Model Artifact for AWS Serving Container ---")
    local_dir = "model_artifact"
    os.makedirs(local_dir, exist_ok=True)

    # Save standard XGBoost binary model artifact
    model_file_path = os.path.join(local_dir, "xgboost-model")
    model.save_model(model_file_path)

    # Compress artifact to model.tar.gz
    tar_filename = "model.tar.gz"
    with tarfile.open(tar_filename, "w:gz") as tar:
        tar.add(model_file_path, arcname="xgboost-model")

    s3_key = "models/xgboost-model.tar.gz"
    s3_uri = f"s3://{BUCKET}/{s3_key}"

    s3_client.upload_file(tar_filename, BUCKET, s3_key)
    print(f"Model successfully archived and uploaded to: {s3_uri}")
    return s3_uri
```

Step 4: Register Candidate Model in SageMaker Model Registry

Register the candidate model inside a **SageMaker Model Package Group** using standard AWS Deep Learning Container (DLC) URIs. Marking `ModelApprovalStatus="Approved"` makes the artifact immediately eligible for deployment pipelines.

Append the registration function and main execution block to complete `src/train_xgboost.py`:

Python

```python
def register_model_package(s3_artifact_uri):
    print("--- Registering Model in SageMaker Model Registry ---")
```

```
    # Ensure Model Package Group exists
    try:
        sm_client.create_model_package_group(
            ModelPackageGroupName=MODEL_PACKAGE_GROUP_NAME,
            ModelPackageGroupDescription="Customer Churn XGBoost Production Model Package Gr
        )
        print(f"Created Model Package Group: {MODEL_PACKAGE_GROUP_NAME}")
    except sm_client.exceptions.ClientError:
        print(f"Model Package Group '{MODEL_PACKAGE_GROUP_NAME}' already exists.")

    # Retrieve standard AWS XGBoost Container Image URI
    image_uri = sagemaker.image_uris.retrieve(
        framework="xgboost",
        region=REGION,
        version="1.5-1"
    )

    create_model_package_input = {
        "ModelPackageGroupName": MODEL_PACKAGE_GROUP_NAME,
        "ModelPackageDescription": "Production XGBoost churn classifier fitted on raw NumPy
        "ModelApprovalStatus": "Approved",
        "InferenceSpecification": {
            "Containers": [
                {
                    "Image": image_uri,
                    "ModelDataUrl": s3_artifact_uri
                }
            ],
            "SupportedTransformInstanceTypes": ["ml.m5.xlarge"],
            "SupportedRealtimeInferenceInstanceTypes": ["ml.m5.xlarge", "ml.t2.medium"],
            "SupportedContentTypes": ["text/csv", "application/json"],
            "SupportedResponseMIMETypes": ["text/csv", "application/json"]
        }
    }

    response = sm_client.create_model_package(**create_model_package_input)
    model_package_arn = response["ModelPackageArn"]
    print(f"Model Package Successfully Registered!")
    print(f"Model Package ARN: {model_package_arn}")
    return model_package_arn

if __name__ == "__main__":
    df = extract_features_from_offline_store()
    model = train_xgboost_model(df)
    s3_uri = package_and_upload_model(model)
    register_model_package(s3_uri)
```

Step 5: Execute and Validate Deliverable 2

Run the Script via Terminal:

```
DOS

python src/train_xgboost.py
```

Expected Output Verification:

```
Plaintext

Querying Offline Store from Feature Group: 'customer-churn-feature-group'...
Data retrieved successfully. Total rows/cols: (7043, 38)
--- Cleaning Schema and Engineering Training Sets ---
Training feature dimension count: 33
--- Training XGBoost Candidate Model ---
XGBoost Training Complete!
--- Packaging Model Artifact for AWS Serving Container ---
Model successfully archived and uploaded to: s3://company-mlops-churn-production-106679
--- Registering Model in SageMaker Model Registry ---
Model Package Successfully Registered!
Model Package ARN: arn:aws:sagemaker:us-east-1:1066795149900:model-package/customer-chu
```

Verify in AWS Console:

Open **Amazon SageMaker Console → Model Registry → Model groups**.

Click `customer-churn-model-group` .

Confirm Version 1 is visible with **Status = Approved**.